

☐ 计算机网络实验报告三-3 滑动窗口的选择确认可靠传输协议

学号:2013365 姓名:颜铭

1. 实验准备

1.1 实验要求

利用数据报套接字在用户空间实现面向连接的可靠数据传输

包含以下基本功能为:

- 建立连接
- 差错校验和包损坏检测
- 不同接收情况的处理, 拥塞控制
- 超时重传, 丢包重传
- 发送窗口和接收窗口采用相同的大小。采用选择确认支持流水线设计
- 日志输出SEQ ACK CHECKSUM以及THROUGHPUT

本次实验在上次没有完善滑动窗口机制的多组发包的基础上

1. 引入了多线程技术并且支持选择重传机制
2. 改进原有的RENO算法, 将 `ackhandler` 的部分逻辑加上有序数组维护(选择确认的实现)
3. 重写计时器触发的代码, 舍弃原来使用的

1.2 前置编程

本次实验使用路由程序作为模拟丢包和设置延时的转发应用, 程序转包最大不超过15000bytes。此外注意先启动路由程序才可以沟通服务器和客户端, 需要将路由器绑定都可用的 `0.0.0.0::<port>` 端口, 设置防火墙规则。

1. 需要在IP和端口设计中需要将客户端 `bind` 到客户端的客户IP (任意一个非路由和服务器端口), 并且通过这个端口接收消息。服务端同理, 将服务端的接收方绑定为服务器IP。无论是客户端还是服务器都要将发送的目标对象的IP设置为路由程序的IP。
2. 服务端使用windows的 `WSA` 套接字编程, 引入 `ioctlsocket` 设置服务器为非阻塞监听, 对 `ANY` 端口的响应进行实时的监听。客户端作为发送方才可以模拟丢包, 因此需要让客户端主动发起握手请求后多一轮确认服务器的握手成功准备发送数据的通信。
3. 需要重新设计 `struct packet`, 至少包含序列号, 校验和, ACK确认以及不同的通信编码标识 (可以等价与FIN, 通过不同tag)。

2. 实验原理和设计

参考TCP在不可靠的网络层服务之上实现可靠传输的各种机制，在应用层添加这些机制，以实现在传输层中不可靠的UDP服务之上提供可靠传输

2.0.1 对于TCP的握手原理

- **第一次握手** 客户端将TCP报文标志位SYN置为1，按通信序列产生一个序号值 $seq = x$ ，保存在TCP首部的序列号(Sequence Number)字段里，指明客户端打算连接的服务器的端口，并将该数据包发送给服务器端，发送完毕后，客户端进入 SYN_SENT 状态，等待服务器端确认。
- **第二次握手** 服务器端收到数据包后由标志位SYN=1知道客户端请求建立连接，服务器端将TCP报文标志位SYN和ACK都置为1， $ack = x + 1$ ，随机产生一个序号值 $seq = K$ ，并将该数据包发送给客户端以确认连接请求，服务器端进入 SYN_RCVD 状态。
- **第三次握手** 客户端收到确认后，检查ack是否为 $x + 1$ ，ACK是否为1，如果正确则将标志位ACK置为1， $ack = K + 1$ ，并将该数据包发送给服务器端，服务器端检查ack是否为 $K + 1$ ，ACK是否为1，如果正确则连接建立成功，客户端和服务器端进入 ESTABLISHED 状态，完成三次握手，随后客户端与服务器端之间可以开始传输数据了。

在确认应答的时候TCP会携带Seq，接收到该报文段的另一端发送确认报文回去，报文中包含确认号（ $ack = seq + 1$ ），为期待接收的下一字节的序号。这样也可以维护发送的报文信息有序

本次实验为简洁使用包的互传数据内容，实际上使用的是 tag 标记了的不同应答标志位策略来模拟了三次握手和四次挥手的环节

2.0.2 TCP的报文结构

我们主要需要维护单向通信的建立连接和分发数据包信息。以及数据段的长度和以 char 为元单位的内容(我们在读取文本和图形的二进制数据时可以需要通过字节流进行内存写入)

我们关注的原理性结构为：

序列号(Seq)：长度为32 bits（4个字节）。指定了当前数据分片中分配给第一字节数据的序列号。在TCP传输流中每一个字节为一个序号。如果TCP报文中flags标志位为SYN，该序列号表示初始化序列号(ISN)，此时第一个数据应该从序列号ISN+1开始。

确认序列号(Ack)：长度为32bits（4个字节）。表示TCP发送者期望接受下一个数据分片的序列号。该序号在TCP分片中Flags标志位为ACK时生效。序列号分片的方向和流的方向同方向，而确认序列号分片方向和流方向反方向。

标志位(tag)：长度为6bits。表示TCP包特定的连接状态。一个标志位占一个bit，从低位到高位值依次为256内的不同状态码，表示不同的请求和应答状态。

校验和(Checksum)：长度16bits（2个字节）。用来检查TCP头在传输中是否被修改。

数据(Data)：长度可变。用来存储上层协议的数据信息,需要设计一个最大的数据存储长度。

窗口(Window)：长度16bits（2个字节）。表示滑动窗口的大小，用来告诉发送端接收端的buffer space的大小。接收端buffer大小用来控制发送端的发送数据速率，从而达到流量控制。最大值为 65535(后置实验中使用)

2.0.3 滑动窗口加上选择确认机制的内容

1. Selective Repeat (SR) 是一种滑动窗口协议，用于实现可靠的数据传输。它是在 UDP 等无连接协议中，通过巧妙地使用窗口机制来实现数据的可靠传输。

基本原理：

1. **滑动窗口：** 发送方和接收方都维护一个窗口，窗口内的每个数据包都有一个唯一的序列号。窗口的大小决定了可以发送或接收的数据包的数量。
2. **序列号：** 每个数据包都有一个唯一的序列号，发送方按照顺序递增的方式为每个数据包分配序列号。接收方使用序列号来正确排序接收到的数据包。
3. **确认和重传：** 发送方发送数据包后，等待接收方的确认。如果在一定时间内未收到确认，发送方会重传对应的数据包。接收方收到数据包后，发送确认，并按序缓存数据。
4. **选择确认：** 在 SR 中，接收方可以选择性地确认收到的数据包。例如，如果接收方收到了序列号为 5 的数据包，但尚未收到序列号为 6 的数据包，它可以选择只确认序列号为 5 的数据包，而不是确认之前的所有数据包。

实现流程：

1. **初始化：** 窗口大小和序列号范围是 SR 协议的关键参数。初始化发送方和接收方的窗口大小，以及初始序列号。
2. **发送数据：** 发送方从应用层获取数据，将数据打包成数据包，分配序列号，然后发送到对端。发送方维护一个发送窗口，跟踪已发送但未确认的数据包。
3. **接收数据：** 接收方接收到数据包后，检查序列号，如果序列号在接收窗口内，表示数据包按序到达，进行确认并将数据交给应用层。如果序列号不在窗口内，表示数据包是之前已确认的重传数据，直接丢弃。
4. **发送方超时处理：** 发送方设置一个定时器，等待接收方的确认。如果定时器超时，触发超时事件，发送方重新发送窗口内的所有未确认数据包。
5. **接收方确认处理：** 接收方收到数据包后，判断是否需要选择性确认。如果只收到序列号为 N 的数据包，但 N+1 还未到达，选择性确认序列号为 N 的数据包。如果接收到的数据包序列号在窗口内，更新窗口，并发送确认。
6. **重传处理：** 如果发送方收到接收方对某个序列号的选择性确认，表明接收方未收到该序列号的数据包，发送方进行重传。

Selective Repeat 算法通过选择性确认和窗口机制，减少了不必要的重传，提高了传输效率。然而，它的实现相对较为复杂，需要处理超时、确认、重传等情况。在实际应用中，需要仔细设计和调整窗口大小、超时时间等参数以达到良好的性能

2.1 实验设计

实验设计客户端的端口为 88881，路由程序的端口为 1099，服务器端的端口为 8888。三者皆为本地回环 IP 地址 127.0.0.1

对于数据包，考虑到与之后的实验兼容要加上端口和预留数据信息。设计为：

```

#define MESSAGELEN 4096
struct packet
{
    unsigned char tag; //连接建立、断开标识
    unsigned int seq; //序列号
    unsigned int ack; //确认号
    unsigned short len; //数据部分长度
    unsigned short len_buffer; //预留数据段位置长度信息。
    unsigned short checksum; //校验和
    unsigned short window; //窗口(停等机制不需要使用)
    char data[MESSAGELEN]; //数据长度
}

```

Fence 2-1

这里的 `seq` 和 `window` 在停等试验中都不是必要的。然后需要确认`ack`和`tag`标识的作用。

同理得到协议设计的架构图为：

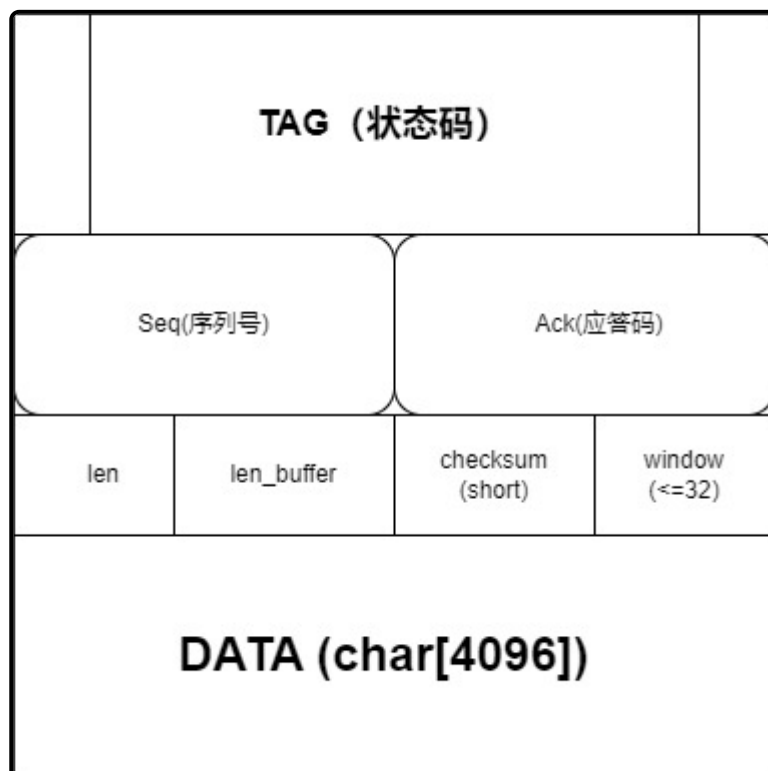


Figure 2-1

校验和算法为：

```

unsigned short make_sum(int count, char* buf)
{
    unsigned long sum = 0;

    while (count)
    {
        sum += *buf;

        if (sum & 0xffff0000)
        {
            sum &= 0xffff;
            sum++;
        }

        buf++;
        count--;
    }

    return ~(sum & 0xffff);
}

```

```
}  
}
```

Fence 2-2

检验部分为：

```
bool corrupt(packet* pkt)
{
    int count = sizeof(pkt->data) / 2;
    register unsigned long sum = 0;
    unsigned short* buf = (unsigned short*)(pkt->data);
    while (count--) {
        sum += *buf++;
        if (sum & 0xFFFF0000) {
            sum &= 0xFFFF;
            sum++;
        }
    }
    if (pkt->checksum == ~(sum & 0xFFFF))
        return true;
    return false;
}
```

Fence 2-3

2.1.1 通信流程

- 握手，首先客户端发送 `tag==1` 的请求握手连接请求。服务端接收后应答100请求后准备接收 `tag==200` 表示允许接受数据。完成三次握手建立连接，然后客户端需要准备数据初始化，使用预留数据段发送文件名 `data`。使得服务器端路径创建文件并打开文件流。
- 挥手，客户端完成发送后发送 `tag==88` 进行广播。服务器端应答188表示检查全部将接受无误后进入挥手阶段，服务器端发送202码，然后客户端接收后发送212码表示请确认关闭连接，服务器端再发送213码表示服务器端预关闭状态，客户端确认 `tag==213` 后进行本地关闭，清空套接字后总共四次挥手。

此外拥塞控制的基本单位实现为 `cwnd` 拥塞窗口和流量阈值 `sssthresh`

3. 实验代码实现

首先考虑实现超时重传机制，初始将ack都设置为1标识期望最小有序待定状态。然后对于返回接受的ack标记为 `ack==3` 而在前面发送时标记为 `ack==2`。超时重传就是在特定的时限后调用函数检查是否已有的单个窗口内序列的ack还是2则认为没有成功传递。在这之前避免占用窗口使用拥塞控制的快速重传实现对临界的ACK应答进行释放。

对于接受不同状态下带拥塞控制的确认，预先设计为慢启动状态。也即在很大的阈值范围内不超过单个串口的单个对ack确认后再触发全局的发送信号开关。

注意:本次实验重写了大部分RENO算法 如下快速重传算法由于不是累积的需要使用单独的序列参数

```
//快速重传
void fastReCoHandler(unsigned int seq)
{

    // lock_guard<mutex>glock(mtx);
    packet* pkt_s = new packet();
    pkt_s->init_packet();
    memcpy(pkt_s, buffer[seq - sendbase], BUFFER);
    sendto(socketClient, (char*)pkt_s, sizeof(*pkt_s), 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));

}
```

Fence 3-1

以及超时重传需要绑定到WINDOWS API的 `SetTimer` 函数作为回调，隐式传入后再通过寄存定时器的数组进行维护。

不同的状态分支中

```
//超时重传
void timeouthandler(HWND hwnd,UINT uMsg,UINT idEvent,DWORD dwTime)
{

    unsigned int seq = -1;

    for (int i=0;i<minwindow(cwnd,WINDOWSIZE);i++)
    {
        if(timerID[i]==idEvent && timerID[i]!=0)
        {
            seq = i + sendbase;//序列号的判定
        }
    }

    if (seq == -1) {
        return; //如果没有超时的，就退出定时器的时间(可能是由于ack处理的问题)
    }

    packet* pkt_s = new packet();
    pkt_s->init_packet();
    memcpy(pkt_s, buffer[seq-sendbase], BUFFER);
    sendto(socketClient, (char*)pkt_s, BUFFER, 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
    cout << "[Repeat-Send (Timeout) " << seq << " 号数据包]" << endl << endl;
    ssthresh = cwnd / 2 == 0 ? 1 : cwnd / 2; //RENO 流量阈值的动态调整
    cwnd = 1; //窗口大小的复位
    STATE = SLOWSTART; //检测到超时，就回到慢启动状态
    cout << "=====检测到超时，回到慢启动阶段
===== " << endl;
    cout << "cwnd= " << cwnd << "      ssthresh= " << ssthresh << endl << endl;

}
```

Fence 3-2

这次将建立三次握手的通讯连接的实现放在两层循环中分别作为抢占性通讯和持久性通信，将多线程放在循环外。

```
while (true)
{
```

[illegible]

```

        break;
    }
}
}

```

Fence 3-3

```

//初始化工作
void init()
{
    //加载套接字库 (必须)
    WORD wVersionRequested;
    WSADATA wsaData;
    //套接字加载时错误提示
    int err;
    //版本 2.2
    wVersionRequested = MAKEWORD(2, 2);
    //加载 dll 文件 Socket 库
    err = WSStartup(wVersionRequested, &wsaData);
    if (err != 0)
    {
        //找不到 winsock.dll
        cout << "WSStartup failed with error: " << err << endl;
        return;
    }
    if (LOBYTE(wsaData.wVersion) != 2 || HIBYTE(wsaData.wVersion) != 2)
    {
        cout << "Could not find a usable version of Winsock.dll" << endl;
        WSACleanup();
    }
    else
    {
        cout << "套接字创建成功" << endl;
    }
    socketClient = socket(AF_INET, SOCK_DGRAM, 0);

    addrServer.sin_family = AF_INET;
    addrServer.sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, SERVER_IP, &addrServer.sin_addr);

    //
    addrClient.sin_family = AF_INET;
    addrClient.sin_port = htons(CLIENT_PORT);
    inet_pton(AF_INET, SERVER_IP, &addrClient.sin_addr);

    err = bind(socketClient, (SOCKADDR*)&addrClient, sizeof(SOCKADDR));
    if (err) {
        err = GetLastError();
        cout << "Could not bind the port" << SERVER_PORT << "for socket. Error code is"
        << err << endl;
        WSACleanup();
        return;
    }
    else
    {
        cout << "客户端注册成功" << endl;
    }

    for (int i = 0; i < WINDOWSIZE; i++)
    {
        ack[i] = 1; //初始标记为1表示最初的序列号
    }
}

```

Fence 3-4

日志设计实现:发送和接受分别实现了对 [Send] 和 [Recv] 状态的检查。

SR算法的实现对比GBN算法在滑动窗口的分组发包中维护一个局部失序修正的块 `[send(recv)_base -N send(recv)_base+N-1]`，使得接收时候可以通过选择确认对失序包缓存并在future中接收正确的顺序发包后交付上层。

对于发送方建立独立运行两个线程，互不干扰可能存在的失序修正以及正常的顺序发包(这样就可以避免对异步任务的生存周期的进一步不可控的问题)：

- 一个发包的循环线程。
- 一个接收验证ACK并执行选择确认

多线程程序引入的WINAPI风格的数据成员：

```
mutex mtx; //资源锁
mutex amtx; //验证锁
mutex signalMtx;
condition_variable cv;
bool packet_ready = false;
bool ack_check[(int)MXN];

HANDLE hThread[2];
int timerID[WINDOWSIZE];
int sendbase = 0;
MSG sysmsg;
```

Fence 3-5

对于接收方缓存失序的数组,单线程地实现选择确认并对与重复的序列号发送同样的ACK。此外要注意平移缓存,这对于发送方的定时器状态数组和全局标记数据也是一样的。

- n在`[recv_base,recv_base+N-1]`的区间中发送ACK(n),缓存失序的数组。按序到达的分组号可以交付上层。
- n在`[recv_base-N,recv_base-1]`的区间中发送ACK(n);

3.0.2 发送方的发包线程

```
DWORD WINAPI sendEvents(LPVOID lparam)
{
    // unique_lock<mutex>glock(mtx);
    // auto sender= static_cast<int*>(lparam);
    // int sendbase = *sender;

    sendbase = 0;
    curseq = 0;

    buffer = new char* [WINDOWSIZE];
    for(int i=0;i<WINDOWSIZE;i++)
    {
        buffer[i] = new char[BUFFER];
    }
    for(int i=0;i<WINDOWSIZE;i++)
    {
        timerID[i] = -1;
    }

    clock_t start = clock();

    auto pkt = new packet();
    while (sendbase < totalpacket) {
```

```

        mtx.lock();

        while ((curseq + seqnumber - sendbase) % seqnumber < WINDOWSIZE && totalack <
totalpacket)
        {

            pkt->init_packet();
            if (lengthFile >= MESSAGELEN)
            {
                is.read(pkt->data, MESSAGELEN);
                make_pkt(pkt, curseq, MESSAGELEN);
                lengthFile -= MESSAGELEN;
            }
            else
            {
                is.read(pkt->data, lengthFile);
                make_pkt(pkt, curseq, lengthFile);
            }

            memcpy(buffer[curseq - sendbase], pkt, BUFFER);
            sendto(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
            cout << "[Send 第 " << curseq << " 号数据包]" << endl << endl;

            timerID[curseq - sendbase] = SetTimer(NULL, 0, TIMEOUT,
(TIMERPROC)timeouthandler);
            cout << "[TimeID -" << curseq << "]: " << timerID[curseq - sendbase] << endl;
            ++curseq;

            curseq %= seqnumber;

            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
        // glock.unlock();

        mtx.unlock();

        while (PeekMessage(&sysmsg, NULL, 0, 0, PM_REMOVE))
        {
            if (sysmsg.message == WM_TIMER)
            {
                DispatchMessage(&sysmsg);
            }
        }
    }
    clock_t end = clock();
    pkt->init_packet();
    pkt->tag = 88;
    sendto(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));

    pkt = new packet;
    pkt->init_packet();
    recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&len);
    int recv_cnt = 0;
    while (recvSize < 0)
    {
        Sleep(100);
        pkt = new packet;
        pkt->init_packet();
        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&len);
        recv_cnt++;

        if (recv_cnt > 100)
        {
            cout << "[Assertion]服务主机未知状态,可选择按下q键离开" << endl;
            Sleep(100);
            char c;

```

```

        while (c = cin.get())
        {
            if (c == 'q')
            {
                ; srOverFlag = true;
            }
        }
    }
}

if (pkt->tag == 188) {
    cout << "*****" << endl;
    cout << "数据传输成功!" << endl;
    cout << "传输用时: " << (end-start) * 1000.0 / CLOCKS_PER_SEC << "ms" << endl;
    cout << "*****" << endl;
    srSendFlag = true;
}

for(int i=0;i<WINDOWSIZE;i++)
{
    KillTimer(NULL, timerID[i]);
}

return 0;
}

```

Fence 3-6

3.0.3 发送方的验证ACK程序

```

DWORD WINAPI recvAckEvents(LPVOID lparam)
{
    // unique_lock<mutex>glock(mtx);
    auto pkt = new packet();

    if (!ackSrFlag)
    {
        ackStart = clock();
        ackSrFlag = true;
    }
    while (sendbase < totalpacket) {

        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&len);
        if(recvSize>0)
        {
            recvPnum += 1;

            mtx.lock();
            ackhandler(pkt->ack);
            mtx.unlock();
            cout << "ACK: " << pkt->ack;
            cout << " CHECKSUM: " << pkt->checksum;
            auto use_time = clock() - ackStart;
            cout << " THROUGHPUT: " << std::fixed << std::setprecision(3) << (min(recvPnum,
sendbase) * MESSAGELEN) / (static_cast<double> (use_time / CLOCKS_PER_SEC)) << " Bytes/s" <<
endl << endl;
        }

    }
    return 0;
}

```

Fence 3-7

接收方的选择确认设计为

```

if (!corrupt(pkt))
{
    if (ack_check[pkt->seq]) {
        make_mypkt(pkt, pkt->seq, recvwindow);
        sendto(sockServer, (char*)pkt, BUFFER, 0, (SOCKADDR*)&addrClient, sizeof(SOCKADDR));
        continue;
    }
    if (pkt->seq < waitseq + WINDOWSIZE && pkt->seq >= waitseq && totalrecv < totalpacket)
    {

        cout << "[Recv 第" << pkt->seq << "号数据包 ]" << endl << endl;
        ack_check[pkt->seq] = true;
        ack_send[pkt->seq - waitseq] = true;
        memcpy(&bufferCache[pkt->seq - waitseq], pkt->data, pkt->len);

        int ack_s = 0;
        totalrecv++;
        while (ack_send[ack_s] && ack_s < WINDOWSIZE)
        {
            out_result.write(bufferCache[ack_s], MESSAGELEN);
            out_result.flush();
            waitseq++; //维护最小失序的序号
            ack_s += 1;
        }

        make_mypkt(pkt, pkt->seq, recvwindow);
        cout << "[Send 第" << pkt->seq << "号数据包的ACK确认]" << endl;

        sendto(sockServer, (char*)pkt, BUFFER, 0, (SOCKADDR*)&addrClient, sizeof(SOCKADDR));
        //向前滑动窗口
        if (ack_s > 0)
        {
            for (int i = 0; i < WINDOWSIZE; i++)
            {
                if (ack_s + i < WINDOWSIZE)
                {
                    ack_send[i] = ack_send[i + ack_s];
                    ack_send[i + ack_s] = false;
                    memcpy(&bufferCache[i], &bufferCache[i + ack_s], pkt->len); //数组平移和局部
缓存失序修正
                    ZeroMemory(&bufferCache[i + ack_s], sizeof(bufferCache[i + ack_s]));
                }
                else
                {
                    ack_send[i] = false;
                    ZeroMemory(&bufferCache[i], sizeof(bufferCache[i]));
                }
            }
        }
    }
    else if (pkt->seq >= waitseq - WINDOWSIZE && pkt->seq <= waitseq - 1)
    {
        ack_check[pkt->seq] = true;

        cout << "[SR: 不在窗口内, 重发一个ACK" << pkt->seq << " ]" << endl << endl;
        make_mypkt(pkt, pkt->seq, recvwindow);

        sendto(sockServer, (char*)pkt, BUFFER, 0, (SOCKADDR*)&addrClient, sizeof(SOCKADDR));
    }
    else
    {
        if(totalpacket==totalrecv)
        {
            pkt->init_packet();
            pkt = connecthandlerClient(188);
            sendto(sockServer, (char*)pkt, BUFFER, 0, (SOCKADDR*)&addrClient,
sizeof(SOCKADDR));

```

```

    }
    globalEnter++;
    if (globalEnter == 10) {
        cout << "[Interruption]: 发送包不在窗口内" << endl << endl;
        globalEnter = 0;
    }
}
}
else
{
    cout << "packet corrupt!not operation!!!" << endl << endl;
}
}

```

Fence 3-8

主线程的并行代码为:

```

mainthread: {
    hThread[0] = CreateThread(NULL, 0, sendEvents, NULL, 0, NULL);
    hThread[1] = CreateThread(NULL, 0, recvAckEvents, NULL, 0, NULL);
    WaitForMultipleObjects(2, hThread, TRUE, INFINITE);

    CloseHandle(hThread[0]);
    CloseHandle(hThread[1]);

    if(srOverFlag)
    {
        goto over;
    }

    if(srSendFlag)
    {
        goto wave;
    }
    else
    {
        cout << "程序已经发送所有文件, 未知错误" << endl << endl;
    }
}
}

```

Fence 3-9

挥手设计的代码实现为:

```

wave:{
    recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
    &length);
    int count_wave = 0;
    while(recvSize<0)
    {
        count_wave++;
        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0,
        (SOCKADDR*)&addrClient, &length);
        if(count_wave>100)
        {
            cout << "[Assertion]服务主机未知状态,可选择按下q键离开" << endl;

            char c;
            while (c = cin.get())
            {
                if (c == 'q')
                {
                    goto over;
                }
            }
        }
    }
}

if(pkt->tag==202)

```

```

    {
        pkt = connecthandlerClient(212);
        sendto(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
    }

    recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&length);
    while(recvSize<0)
    {

        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0,
(SOCKADDR*)&addrClient, &length);

    }
    if(pkt->tag==213)
    {
        cout << "#####" << endl;
        cout << "挥手成功, 断开连接完成" << endl;
        cout << "#####" << endl;
        goto over;
    }

}
over:{
closesocket(socketClient);
WSACleanup();

exit(0);
}

```

Fence 3-10

4. 实验结果分析

路由程序的设置界面为

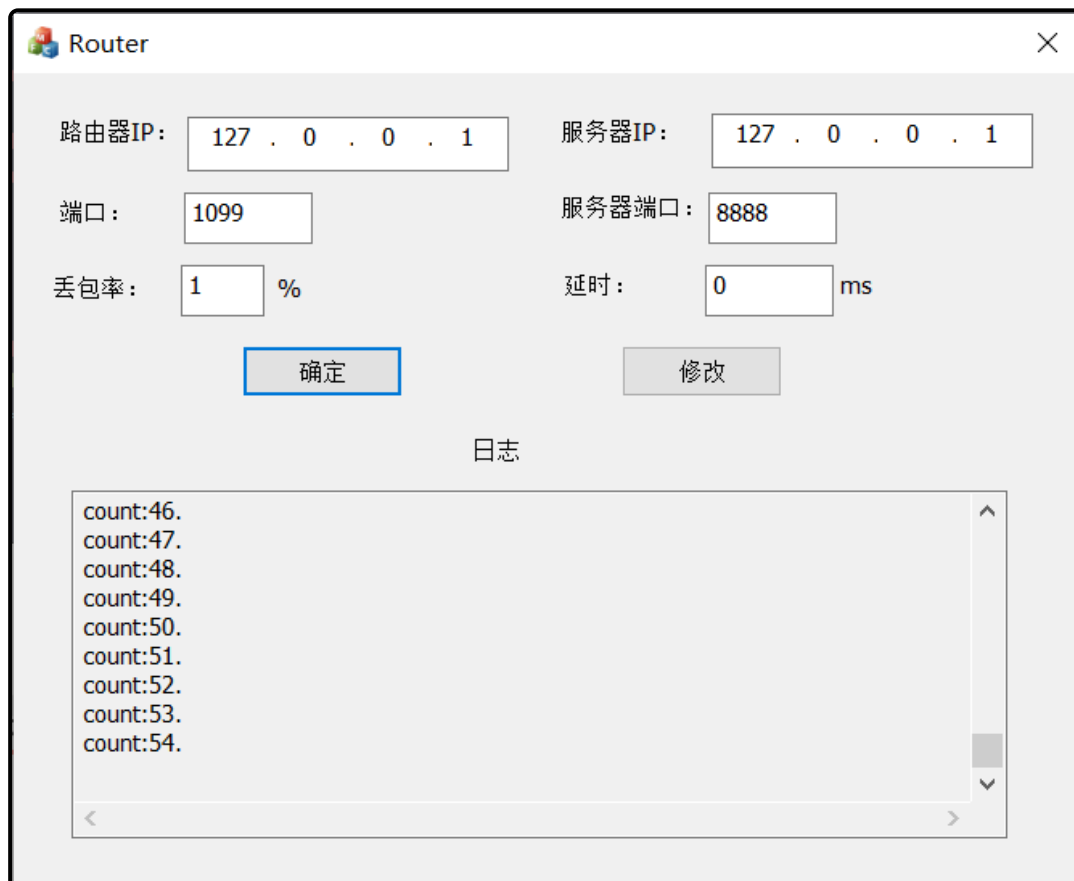


Figure 4-1

有程序运行结果截图为：

建立握手请求

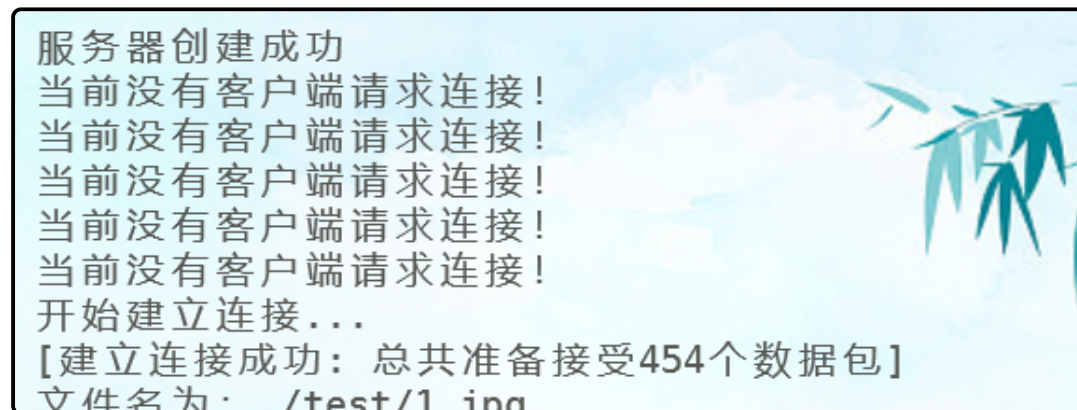


Figure 4-2



Figure 4-3

4.1 服务器端（接收方）

4.1.1 第一个文件：1.jpg



Figure 4-4

4.1.2 第二个文件 2.jpg



Figure 4-5

4.1.3 第三个文件:



Figure 4-6

4.2 客户端(发送方)

4.2.1 1.jpg



```
[TimeID -53]: 13727
[Repeat-Send (TimeOut) 38 号数据包]

=====检测到超时，回到慢启动阶段=====
=====
cwnd= 1.000      sstresh= 0

ACK: 37 CHECKSUM: 65507 THROUGHPUT: 155648.000 Bytes/s

[Repeat-Send (TimeOut) 38 号数据包]

=====检测到超时，回到慢启动阶段=====
=====
cwnd= 1.000      sstresh= 0

[Recv 第38号数据包的ack]

ACK: 38 CHECKSUM: [Send 第 54 号数据包]
65428 THROUGHPUT: 159744.000 Bytes/s
[TimeID -54]: 13726
```

Figure 4-7

4.2.2 2.jpg



```
ACK: 59 CHECKSUM: 65445 THROUGHPUT: 245760.000 Bytes/s
[Send 第 75 号数据包]

[TimeID -75]: 12142
ACK: 75 CHECKSUM: 41 THROUGHPUT: 245760.000 Bytes/s[Repeat-Send (TimeOut)
60 号数据包]

=====检测到超时，回到慢启动阶段=====
=====
cwnd= 1.000      sstresh= 0

[Recv 第60号数据包的ack]

ACK: 60 CHECKSUM: 61 THROUGHPUT: 249856.000 Bytes/s

[Send 第 76 号数据包]

[TimeID -76]: 12141
```

Figure 4-8

4.2.3 Helloworld.txt



Figure 4-9

丢包验证停等如下，延时设计为15s。以及重传触发

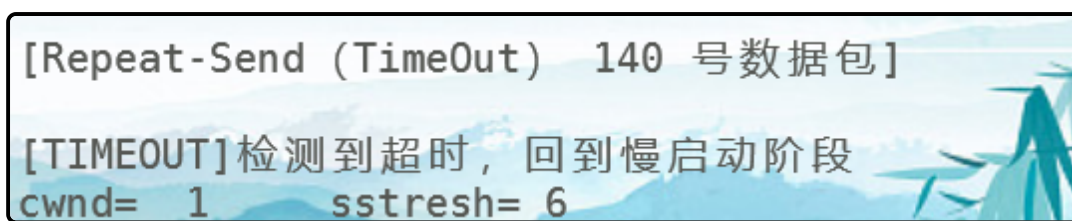


Figure 4-10

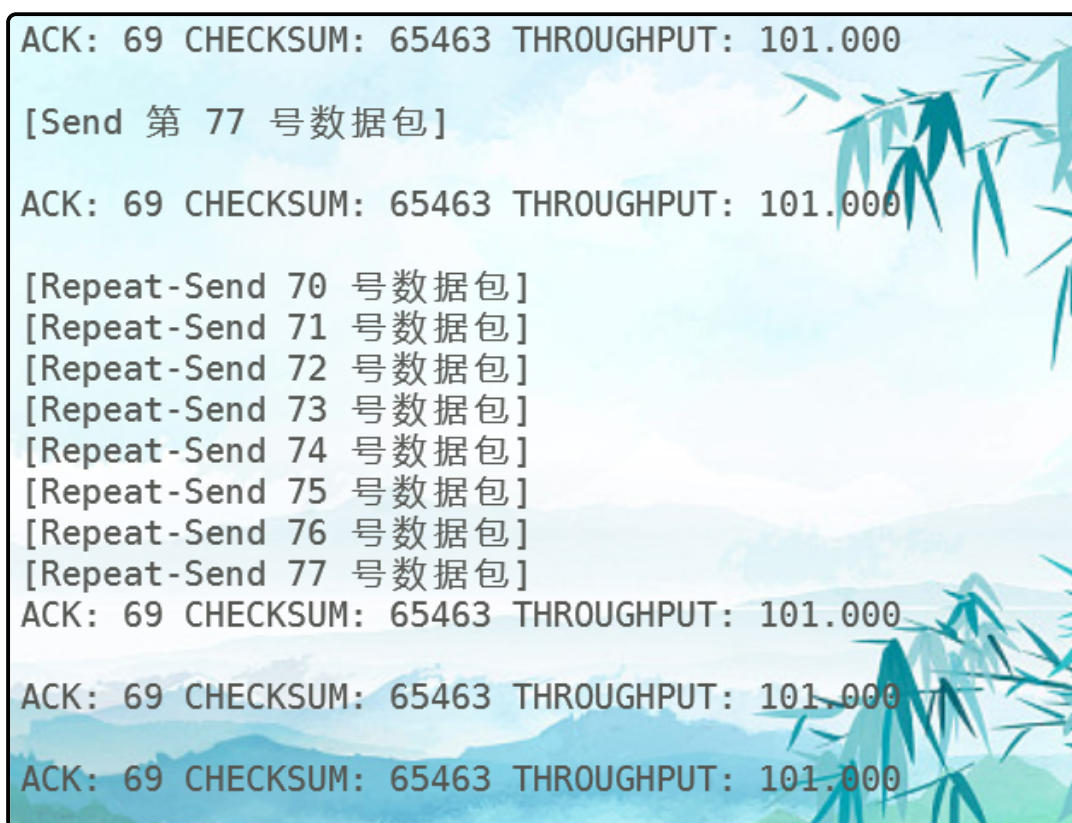


Figure 4-11

握手成功:

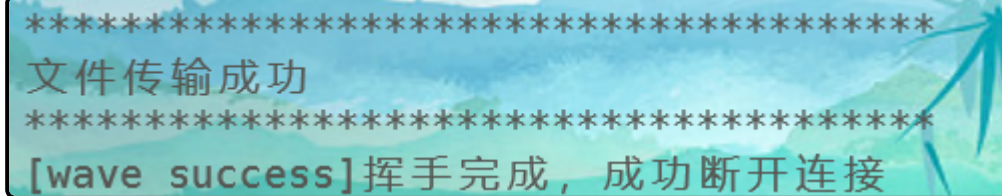


Figure 4-12



Figure 4-13

5. 实验问题解答

5.1 实验总结:

本次改进了上次GBN实现中未完善的双线程编程任务。并且通过更多种失序和待接收包的状态分支维护了正确实现选择确认

归纳以下选择确认特点，从滑动窗口的运行状态出发对比GBN算法的特点为：

- 1. 确认机制：** SR 使用选择性确认机制。接收方可以选择确认按序到达的每个数据包，而不需要等待中间的非最后一个数据包。
- 2. 重传机制：** 如果发送方发送的某个数据包未收到确认，只重传这个未确认的数据包，而不是窗口内的所有未确认的数据包。
- 3. 效率：** SR 允许发送方在窗口内的数据包中选择性地重传，因此在一定程度上提高了效率。发送方不必因为某个数据包的丢失而重传整个窗口。

改进点：

- 1. 选择性确认：** SR 具有选择性确认的能力，可以确认按序到达的每个数据包，而 GBN 只确认最后一个数据包。这意味着 SR 在收到的确认中更灵活，可以避免一些不必要的重传。

2. **灵活的重传：** SR 允许发送方灵活地选择性重传未收到确认的数据包，而 GBN 必须重传整个窗口。这减少了在网络中的冗余传输。
3. **发送方不阻塞：** 在 GBN 中，如果窗口内的某个数据包未得到确认，发送方将被阻塞。而在 SR 中，发送方可以继续发送其他数据包，不受未确认数据包的阻碍。

总体而言，SR 算法相对于 GBN 具有更好的灵活性和效率，特别是在面对不可靠的信道时。然而，SR 也带来了一些额外的复杂性，包括序列号的管理和选择性确认的处理。这就可能导致额外的空间复杂度。

5.2 实验不足

2.jpg 中存在程序不能正确进入挥手程序的 bug，但文件整体传输正确，且与 WINDOWSIZE 和 TIMEOUT 的设置有关